

Question 1:

Create a function to reverse a list in Python. Use an AI tool to suggest alternative implementations. Test and compare the performance of each version.

(Each question carries 5 marks)

Prompt Used:

```
#Create a function to reverse a list in Python. Use different types of Implementations
```

Code:

```
Q1.py
1  #Create a function to reverse a list in Python. Use different types of Implementations
2
3  # Implementation 1: Using slicing
4  def reverse_list_slicing(lst):
5      """
6      Reverse a list using slicing.
7
8      Args:
9      |   lst (list): The list to reverse.
10
11     Returns:
12     |   list: Reversed list.
13     """
14     return lst[::-1]
15
16
17 # Implementation 2: Using built-in reversed()
18 def reverse_list_reversed(lst):
19     """
20     Reverse a list using the built-in reversed() function.
21
22     Args:
23     |   lst (list): The list to reverse.
24
25     Returns:
26     |   list: Reversed list.
27     """
28     return list(reversed(lst))
29
```

```

0
1  # Implementation 3: Using a for-loop
2  def reverse_list_loop(lst):
3      """
4      Reverse a list using a for-loop and insert().
5
6      Args:
7      |     lst (list): The list to reverse.
8
9      Returns:
10     |     list: Reversed list.
11     """
12     reversed_lst = []
13     for item in lst:
14         reversed_lst.insert(0, item)
15     return reversed_lst
16
17
18  # Implementation 4: In-place swap
19  def reverse_list_inplace(lst):
20      """
21      Reverse a list in place using two-pointer swap.
22
23      Args:
24      |     lst (list): The list to reverse.
25
26      Returns:
27      |     list: Reversed list.
28      """
29      left, right = 0, len(lst) - 1
30      while left < right:
31          lst[left], lst[right] = lst[right], lst[left]
32          left += 1
33          right -= 1
34      return lst
35
36
37  # Test all implementations
38  if __name__ == "__main__":
39      my_list = [1, 2, 3, 4, 5]
40
41      print("Original list:", my_list)
42      print("Slicing:", reverse_list_slicing(my_list))
43      print("Reversed():", reverse_list_reversed(my_list))
44      print("For-loop:", reverse_list_loop(my_list))
45      print("In-place:", reverse_list_inplace(my_list[:])) # copy for in-place
46

```

Output:

```

PS C:\Users\skondaparthi\OneDrive - Innominds Software Private Limited\Desktop\lab_test_1> python Q1.py
Original list: [1, 2, 3, 4, 5]
Slicing: [5, 4, 3, 2, 1]
Reversed(): [5, 4, 3, 2, 1]
For-loop: [5, 4, 3, 2, 1]
In-place: [5, 4, 3, 2, 1]
PS C:\Users\skondaparthi\OneDrive - Innominds Software Private Limited\Desktop\lab_test_1>

```

Comparision among the different Implementations:

Implementation	Differences from Others	Advantages	Disadvantages	Possible Improvements
1. Slicing	Creates a new reversed list using Python slicing. Simple one-liner.	Very fast, concise, easy to read.	Creates an extra list (uses extra memory).	Could add in-place version to save memory.
2. reversed()	Uses Python built-in iterator; unlike slicing, returns an iterator until converted.	Clear, Pythonic, fast, flexible since it can be used in loops directly.	Needs <code>list()</code> to get an actual list; slightly more steps than slicing.	Could use <code>itertools</code> for more advanced use cases.
3. For-loop with	Manually inserts elements at the front, unlike slicing or <code>reversed()</code> .	Conceptually simple, good for teaching how reversal works internally.	Very slow for large lists ($O(n^2)$), inefficient memory-wise.	Replace <code>insert(0,...)</code> with <code>append</code> and then <code>reverse</code> at the end for better efficiency.
4. In-place swap	Reverses the list by swapping elements in place. Unlike others,	Memory is not efficient	Modifies original list; slightly more complex logic than slicing.	Could add a copy option to preserve the original list.

	modifies the original list.			
--	-----------------------------	--	--	--

Question 2:

Design a few-shot prompt to guide an AI tool in writing code for a basic linked list with methods for insert and delete. Analyze the resulting code for correctness and completeness.

Prompt Used:

#You are a Python programming assistant. Write Python classes for common data structures. Each example should include:

- #A clear class definition
- #Methods implementation
- #A small usage example
- #Proper handling of edge cases

"""Example 1: Stack

```
class Stack: def init(self): self.items = []
```

```
def push(self, item):
    self.items.append(item)
```

```
def pop(self):
    if self.items:
        return self.items.pop()
    return None
```

```
stack = Stack() stack.push(10) stack.push(20) print(stack.pop()) # 20
```

Example 2: Queue

```
class Queue: def init(self): self.items = []
```

```
def enqueue(self, item):
    self.items.append(item)

def dequeue(self):
    if self.items:
        return self.items.pop(0)
    return None

queue = Queue() queue.enqueue(1) queue.enqueue(2) print(queue.dequeue()) # 1
```

Now, write Python code for a **Singly Linked List** with the following specifications:

1. `insert(data)`: Insert a new node at the end of the list.
2. `delete(value)`: Delete a node by its value.
3. `display()`: Print all nodes in order.

Include a usage example demonstrating insertion, deletion, and display. Make sure the code handles edge cases like deleting from an empty list or deleting a value not present in the list. """

Code:

```

46
47 # Node class for singly linked list
48 class Node:
49     def __init__(self, data):
50         self.data = data
51         self.next = None
52
53 # Singly Linked List class
54 class SinglyLinkedList:
55     def __init__(self):
56         self.head = None
57
58     def insert(self, data):
59         """Insert a new node at the end of the list."""
60         new_node = Node(data)
61         if not self.head:
62             self.head = new_node
63             return
64         current = self.head
65         while current.next:
66             current = current.next
67         current.next = new_node
68
69     def delete(self, value):
70         """Delete the first node with the specified value."""
71         current = self.head
72         prev = None
73
74         while current and current.data != value:
75             prev = current
76             current = current.next
77

```

```

78         if not current:
79             print(f"Value {value} not found in the list.")
80             return
81
82         if prev is None:
83             # Deleting the head node
84             self.head = current.next
85         else:
86             prev.next = current.next
87
88     def display(self):
89         """Print all nodes in the list."""
90         current = self.head
91         while current:
92             print(current.data, end=" -> ")
93             current = current.next
94         print("None")
95
96
97     # Usage example
98     if __name__ == "__main__":
99         ll = SinglyLinkedList()
100         ll.insert(10)
101         ll.insert(20)
102         ll.insert(30)
103         ll.display() # 10 -> 20 -> 30 -> None
104
105         ll.delete(20)
106         ll.display() # 10 -> 30 -> None
107
108         ll.delete(100) # Value 100 not found in the list
109

```

Output:

```

PS C:\Users\skondaparathi\OneDrive - Innominds Software Private Limited\Desktop\lab_test_1> python Q2.py
10 -> 20 -> 30 -> None
10 -> 30 -> None
Value 100 not found in the list.
PS C:\Users\skondaparathi\OneDrive - Innominds Software Private Limited\Desktop\lab_test_1>

```

Analysis for Completeness and Correctness:

- The insert method correctly adds nodes at the end and handles empty lists.
- The delete method safely removes nodes, including the head, and prints a message if the value is not found.

- The `display` method traverses the list and prints all nodes clearly.
- Edge cases, like deleting from an empty list or a non-existent value, are handled without errors.
- The code structure is clear, with separate `Node` and `SinglyLinkedList` classes and readable methods.
- All requested functionalities are implemented and tested in the usage example.
- Minor improvements could include returning a status from `delete()` and adding `search()` or `insert_at_index()` methods.